VENDOPOS

Complete System Documentation

Multi-tenant Cloud POS + ERP platform built for the Philippines

Architecture, modules, data model, API reference, security, compliance (BIR / VAT), subscription tiers, hardware integration, deployment and operations — documented directly from the working source code.

Product	**VendoPOS** — Point of Sale + Enterprise Resource Planning
Market	Philippine retail, food & beverage, and small manufacturing
Stack	Next.js 16 · React 19 · Tailwind CSS v4 · Express · TypeScript · PostgreSQL (NeonDB)
Repository	emciiowhy/VENDO
Document date	2 October 2026
Version	1.0 — reflects backend `vendopos-backend 0.1.0`

Contents

1. Executive Summary
2. Domain Language & Actors
3. System Architecture
4. Technology Stack
5. Subscription Plans & Feature Gating
6. Tenant Lifecycle & Trials
7. Authentication, Roles & Sessions
8. Point of Sale (POS) Terminal
9. Checkout Engine, VAT & Invoicing
10. Voids, Returns & POS Audit Log
11. Cashier Shifts (X-Read / Z-Read)
12. Inventory & Stock Ledger
13. Procurement & Supply Chain
14. Manufacturing (Bill of Materials)
15. Finance & Accounting
16. Human Resources & Payroll
17. Customer Relationship Management
18. Merchant Dashboard, Analytics & Forecasting
19. BIR Compliance
20. Account, Store Settings & Theme Studio
21. Notifications & Real-Time Streams
22. Super Admin Platform Console
23. Marketing Website & Lead Capture
24. Hardware Integration
25. Database Schema Reference
26. Complete API Reference
27. Frontend Route Map
28. Security Model
29. Installation & Configuration
30. Testing & Quality Assurance
31. Staging Validation Runbook
32. Capability Assessment & Roadmap

1. Executive Summary

VendoPOS is a multi-tenant Software-as-a-Service platform that combines a cloud **Point of Sale** with a full **Enterprise Resource Planning** back office. One platform operator (the Super Admin) sells subscriptions to many independent businesses (Merchants). Every Merchant runs on its own isolated **Tenant**: its products, sales, staff, customers and finances are fenced off from every other store.

The platform is designed from the ground up for the Philippine market: prices are in Philippine peso (₱), VAT is the 12% price-inclusive BIR model, invoices carry gap-free BIR-style serial numbers, receipts carry machine-accreditation fields (PTU, MIN, serial), e-wallet rails **GCash, Maya and QR Ph** are first-class payment methods, and payroll applies Philippine overtime and night-differential norms in Manila time.

What the system does

FRONT OF HOUSE

- Touch POS terminal with cart, discounts, loyalty redemption and split tender channels
- Barcode scan-to-cart (keyboard-wedge scanners)
- Scan-to-pay QR for GCash / Maya / QR Ph
- Customer-facing second display
- HTML and ESC/POS thermal receipts (USB & Bluetooth)
- Shared-terminal cashier PIN switching, shift X-Read / Z-Read

BACK OFFICE (ERP)

- Inventory with an immutable stock-movement ledger
- Procurement: suppliers, purchase orders, receiving, reorder suggestions
- Manufacturing: recipes / bill of materials and production runs
- Finance: expenses, P&L, Balance Sheet, Cash Flow
- HR: employees, attendance, timecards, payroll runs & CSV export
- CRM: customers and loyalty points

COMPLIANCE & CONTROL

- Serialized invoices per tenant (`SI-000123`), locked under concurrency
- Monthly BIR sales ledger (JSON + CSV)
- Voids/returns as contra-entries; POS audit log of sensitive actions
- Cash-drawer variance flags to the centavo

PLATFORM OPERATIONS

- Lead pipeline and atomic lead → tenant provisioning
- Self-serve sign-up with a 14-day trial
- MRR / ARPA billing analytics, subscriber register
- Live system-health stream (SSE), cross-tenant compliance register
- Support impersonation of a merchant owner

Key engineering principles

- **Money is integer centavos end-to-end.** No floating-point peso arithmetic is persisted; VAT is computed once at checkout and frozen.
- **Tenant scope comes from the session, never the request body.** Every query is threaded with the tenant id taken from the signed session cookie.
- **ACID by default.** Checkout, voids/returns, receiving, production, payroll and provisioning are each single all-or-nothing transactions.
- **Server is the source of truth for prices.** The terminal only sends product ids and quantities; the server re-derives every price.
- **Pure, IO-free calculation cores** (payroll, financial reports, reorder, forecast, theme) are unit-tested in isolation.
- **Manila time** (UTC+8) defines "today", monthly buckets and payroll days so figures line up with the local retail day.

2. Domain Language & Actors

VendoPOS maintains a strict ubiquitous language (see `CONTEXT.md`). The same words are used in code, UI and documentation.

Term	Meaning	Avoid
Super Admin	The owner-operator of VendoPOS itself. Sees everything across every Tenant, manages subscriptions, sits above all Tenants. Has no tenant id.	Platform admin, root, developer
Merchant	A business that subscribes to VendoPOS to run its own point of sale and operations — the tenant viewed from the business side.	Customer, client, account
Tenant	The isolated data-and-configuration boundary that belongs to one Merchant — the same thing named from the architecture side.	—
Customer	A shopper buying goods or services from a Merchant at the till.	Using "customer" for the subscribing business
Lead	A visitor who submitted the "Request a Demo" form but is not yet a Tenant. Converted by the Super Admin.	Sign-up, registrant

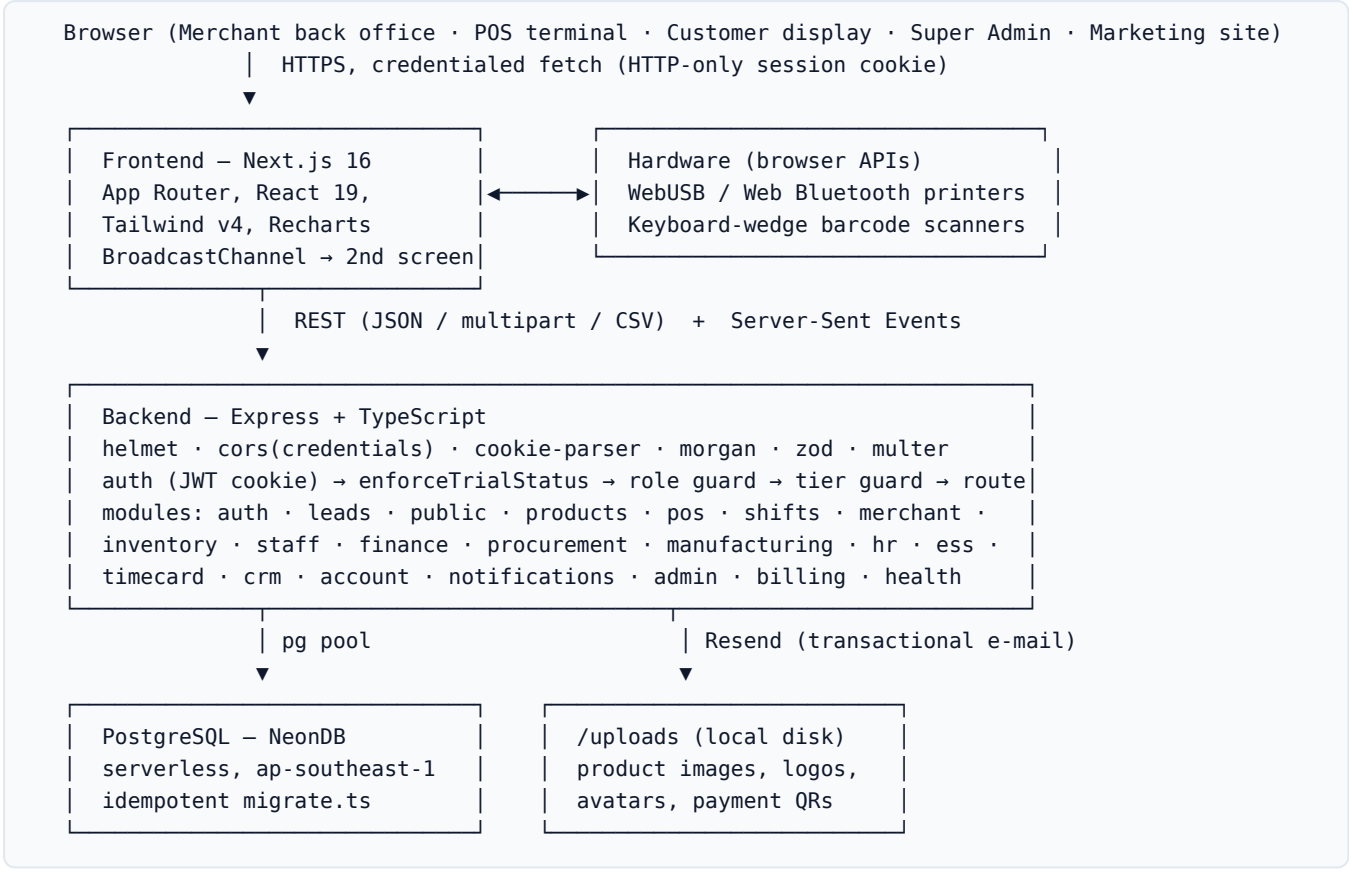
User roles

Role	Scope	Sign-in method	Landing page
`SUPER_ADMIN`	Platform-wide; no tenant (enforced by DB check)	Google OAuth	`/admin`
`MERCHANT_OWNER`	One tenant; full control incl. staff, PINs, deactivation	Google OAuth or Store ID + password	`/dashboard`
`MANAGER`	One tenant; back office except owner-only actions	Google OAuth or Store ID + password	`/dashboard`
`CASHIER`	One tenant; POS terminal, shifts, timecard, CRM lookup	Store ID + 4-digit PIN	`/pos`

ERP modules — one idea each

Module	Owns
Point of Sale	Ringing up a Customer and taking payment.
Inventory	What stock a tenant *has* and where it sits.
Procurement & Supply Chain	How goods *arrive*: suppliers, purchase orders, receiving.
Manufacturing	How goods are *produced*: recipes / bill of materials.
Finance & Accounting	Money in and out: sales reports, expenses, tax, P&L, statements.
Human Resources	Employees, shifts, attendance and payroll.
CRM	Tracking Customers and loyalty over time.

3. System Architecture



Request pipeline

1. **helmet** sets security headers; **CORS** allows the configured origins with `credentials: true` and the verbs GET, POST, PUT, PATCH, DELETE, OPTIONS.
2. JSON bodies are capped at **100 kB**; cookies are parsed for the session JWT.
3. Public seams mount first: `/health`, `/uploads`, `/auth`, `/api/v1/public`, `/api/leads`.
4. **Trial checkpoint** (`enforceTrialStatus`) fences a lapsed-trial store out of every authenticated merchant module with `402` and a recovery marker. It is a no-op for anonymous and Super Admin requests.
5. Each module router applies `requireRole(...)` and, where premium, `requireFeature(...)`.
6. Unknown routes return `404 {"ok":false,"error":"Not found"}`.

Backend module layout

Each module follows a **router → schema (zod) → repository (SQL)** pattern, with pure helpers alongside and a vitest file per module.

Folder	Responsibility
`auth/`	Google OAuth, password & PIN login, JWT, sessions, impersonation, sign-up, role and tier middleware, demo seed
`leads/`, `public/`	Demo / enterprise lead capture, public tenant directory
`products/`, `inventory/`	Catalog, categories, image upload, stock ledger, movements, ledger drift
`pos/`	Catalog feed, checkout, sales, voids/returns, shifts, POS audit log

`merchant/`	Dashboard analytics, forecast, compliance export, SSE events, payment QRs, theme config
`staff/`	Cashier roster, PIN administration, remote shift close
`finance/`, `procurement/`, `manufacturing/`, `hr/`, `crm/`	ERP pillars
`account/`, `notifications/`	Store/profile settings, sessions, exports; in-app notifications and e-mails
`admin/`, `billing/`, `health/`	Super Admin leads, tenants, compliance, MRR, trial engine, health stream

4. Technology Stack

Layer	Technology	Notes
Frontend framework	Next.js 16.2 (App Router), React 19.2	`frontend/`, runs on port 3000
Styling	Tailwind CSS v4, design tokens	Light / dark themes; tenant theme tokens injected as scoped styles (ADR 0001: redesign within the token system)
Charts	Recharts 3	MRR line, plan donut, fee bars, finance and labor charts
Backend	Express 4, TypeScript 5, ES modules	`backend/`, runs on port 4000; `tsx` for dev
Validation	zod	Every write payload is schema-validated; `400 {errors}` on failure
Database	PostgreSQL via NeonDB (serverless), `pg` driver	Idempotent schema in `src/migrate.ts`
Auth	Google OAuth 2.0, `jsonwebtoken`, Node `crypto.scrypt`	HTTP-only cookie `vendopos_session`, 7-day max age
Uploads	multer	Images ≤ 2 MB, served from `/uploads`
E-mail	Resend	Optional — without `RESEND_API_KEY` messages are logged
Security headers / logs	helmet, morgan	
Testing	vitest, supertest; Playwright (screenshots)	~288 backend test cases

5. Subscription Plans & Feature Gating

Two vocabularies are kept in lockstep: the lowercase billing **plan** (`starter / business / enterprise`) drives price, MRR and product capacity; the uppercase **tier** (`STARTER < BUSINESS < ENTERPRISE`) drives feature gating. A tenant's tier is always projected from its plan (`tierFromPlan`), so they can never drift.

Plan catalogue (backend `billing/plans.ts`)

Plan	Billing price / month	Product capacity	Target
Starter	₱499.00 (49,900 centavos)	30 products	Single-location coffee shops, small retailers
Business	₱1,499.00 (149,900 centavos)	300 products	Growing multi-staff retailers and restaurants
Enterprise	₱7,999.00 (799,900 centavos) for MRR; "Contact us" publicly	Unlimited	Full ERP, manufacturing, HR, multi-location

> **Pricing note.** The public marketing pricing page currently advertises **₱599** (Starter) and **₱1,999** (Business), while the backend MRR engine uses ₱499 and ₱1,499. Both are marked as indicative placeholders pending finalisation — align `billing/plans.ts` with `Pricing.tsx` once prices are final.

Exceeding capacity on `POST /api/inventory/products` returns `403 {"code":"plan_limit_exceeded"}`. The platform also models a **0.5%** transaction-fee rate on merchant GMV for fee-revenue charts.

Feature matrix (backend `lib/tiers.ts`, mirrored in `frontend/lib/tiers.ts`)

Feature key	Capability	Minimum tier
`pos_terminal`	Core POS terminal	Starter
`inventory_basic`	Catalogue & inventory management	Starter
`bir_receipts`	BIR-compliant receipts and serial invoicing	Starter
`digital_payments_qrph`	QR Ph / e-wallet payments	Starter
`offline_mode`	Offline-capable register (declared; see roadmap)	Starter
`procurement_supply_chain`	Suppliers and purchase orders	Business
`customer_relationship_crm`	Customers and loyalty	Business
`finance_accounting`	Expenses, P&L, statements	Business
`multi_staff_roles`	Managers and multiple staff roles	Business
`custom_branding`	Theme Studio custom themes	Business
`manufacturing_bom`	Recipes / bill of materials	Enterprise
`human_resources_payroll`	HR roster, attendance, payroll, ESS	Enterprise
`multi_location`	Multiple store locations (declared; see roadmap)	Enterprise
`priority_support`	Priority support SLA	Enterprise
`predictive_inventory`	Predictive reorder forecasting	Enterprise

The API guard `requireFeature(feature)` rejects lower tiers; the frontend `<FeatureGate>` and the merchant navigation hide or upsell exactly what the API forbids.

6. Tenant Lifecycle & Trials

Each tenant row carries `status ∈ {trial, active, trial_expired, suspended}` plus `trial_starts_at` / `trial_ends_at`.

Two onboarding paths

PATH A — SELF-SERVE SIGN-UP

`POST /auth/signup` from the pricing page creates a fresh, isolated tenant on the chosen plan, opened on a **14-day trial**, with the owner account and an onboarding e-mail.

PATH B — SALES-LED (LEAD)

The demo form writes a Lead (`PENDING_DEMO`). The Super Admin approves it, which **atomically** inserts the tenant, inserts the owner as `MERCHANT_OWNER` and flips the lead to `APPROVED`. Slug or owner-email collisions roll back with `409`.

Lazy trial expiry (no cron)

1. On any request after `trial_ends_at`, `evaluateTenantStatus` runs a single guarded `UPDATE … WHERE status='trial' AND trial_ends_at <= now()`.
2. Only the request that wins the flip sends the **trial-expired recovery e-mail** — so it is sent exactly once, even under concurrency.
3. The trial middleware then returns `402` for merchant modules; `/auth/me` reports the lapsed state so the workspace swaps to the **Trial Recovery** view.
4. Sign-in, `/auth/me` and the marketing site stay reachable so the owner can upgrade and recover.

Lifecycle is resolved live from the database, never from the JWT, so an upgrade or lapse takes effect on the very next request. The Super Admin can change a tenant's plan or status via `PATCH /api/v1/admin/tenants/:id`.

7. Authentication, Roles & Sessions

Sign-in methods

Method	Flow
Google OAuth	`GET /auth/google` mints a CSRF `state` and redirects to Google. `/auth/google/callback` verifies state, exchanges the code, looks up the e-mail in `users`, signs the JWT, sets the cookie and redirects to the role's dashboard. Unknown e-mails get `?error=not_provisioned`.
Owner / manager password	`POST /auth/password-login` with Store ID + e-mail + password. Owners set or change a password via `/auth/password`; status via `/auth/password/status`. A first-login onboarding card prompts owners to set one.
Cashier PIN	`POST /auth/store` resolves the Store ID (tenant slug); `POST /auth/pin` verifies a 4-digit PIN **within that tenant** (optionally pinned to a selected `userId`) and swaps the cookie. On a warm terminal the active session's tenant always wins, so a typed Store ID can never reach across tenants.

Forgot-PIN workflow

Cashiers never set their own PIN. `POST /auth/cashiers` lists a store's cashier names; `POST /auth/pin/forgot` raises one pending reset request per cashier (partial unique index, idempotent). The owner resolves it in **Cashiers** by setting a new PIN (resolves the request in the same transaction) or rejecting it.

Sessions & security

- JWT in an HTTP-only cookie (`vendopos_session`), 7-day lifetime, optional `COOKIE_DOMAIN` for split subdomains.
- Each sign-in creates a row in `sessions` (revocable device sessions) and `login_events` (login audit). Users can revoke a single session or all others.
- PINs hashed as `scrypt$salt$hash`; passwords as `pw1$salt$hash` (scrypt, per-user salt); verification is constant-time (`timingSafeEqual`).
- `GET /auth/me` returns `{userId, tenantId, role, email, name, tenantName, …}` with the tenant name, tier, status and theme looked up live.
- **Impersonation:** `POST /auth/impersonate` (Super Admin) mints an owner session for support, shown with an **Impersonation Banner**; `/auth/impersonate/stop` returns to `/admin/tenants`.
- `POST /auth/logout` clears the cookie. All sign-out and terminal-lock actions go through a confirm dialog.

8. Point of Sale (POS) Terminal

Route `/pos`. Available to `CASHIER`, `MANAGER` and `MERCHANT_OWNER`. On a fresh device an **Activation Wizard** binds the terminal to a store.

Terminal features

- **Live catalog** (`GET /api/v1/pos/catalog`): store name, categories and active products with price, stock, low-stock threshold and image; category filter and search.
- **Search or scan barcode:** an Enter-terminated burst resolves to an exact SKU (then a sole-result fallback), drops it in the cart and clears for the next scan; transient "not found / out of stock" feedback. A global keyboard-wedge hook distinguishes scanner bursts from human typing by inter-key timing.
- **Cart**: quantity edits, line voids (logged), cart-level discount (percent 0–100 or fixed amount) with label.
- **Customer attach & loyalty**: search CRM customers, show balance, redeem points.
- **Payment**: Cash (tendered and change), GCash, Maya, QR Ph (reference code). E-wallet rails show the merchant's uploaded **scan-to-pay QR** on the pad and the customer display.
- **Receipts**: printable HTML receipt and direct ESC/POS thermal printing; reprint from sales history.
- **Sales history & returns** modal: list sales, view details, void or return lines.
- **Shift controls**: open shift with float, X-Read, close (Z-Read), open-drawer (logged).
- **Cashier switch**: PIN pad with profile selector for shared terminals; terminal lock.
- **Clock widget**: staff clock-in / clock-out (timecards).
- **Theming**: the tenant theme and announcement bar apply to the terminal.

Customer-Facing Display

Route `/pos/customer-display`: a clean, high-contrast second screen mirroring the live cart, discounts and totals. Sync is purely client-side over a same-origin `BroadcastChannel` (no server round-trip), so it is isolated to the active terminal browser. Selecting an e-wallet rail animates a QR presentation sheet open on the customer screen.

9. Checkout Engine, VAT & Invoicing

`POST /api/v1/pos/orders` is a single ACID transaction.

Request

```
{
  "items": [{ "productId": "uuid", "qty": 2 }],
  "paymentMethod": "Cash" | "GCash" | "Maya" | "QRPH",
  "tenderedCents": 50000,            // cash only
  "referenceCode": "GC-123…",        // e-wallet reference
  "discount": { "type": "percent" | "fixed", "value": 10, "label": "Senior" },
  "customerId": "uuid",              // optional CRM link
  "redeemPoints": 50                 // optional loyalty redemption
}
```

Steps inside the transaction

1. Lock each product row `SELECT … FOR UPDATE`; re-derive prices from the DB (client prices are ignored).
2. Abort with `400` (+ `productId`, `available`) if any line lacks stock.
3. Apply the cart discount server-side (percent 0–100 or fixed centavos).
4. Redeem loyalty: **1 point = ₱1**, clamped to the live balance and to what remains payable.
5. Compute VAT on the **net**: `vat = round(net × 12 / 112)`, `subtotal = net − vat` (12% price-inclusive).
6. Earn loyalty on what was actually paid: **1 point per ₱100** of net.
7. Lock the per-tenant `invoice_counters` row and assign a **gap-free, chronological** reference `SI-000123` (unique index backstop).
8. Decrement stock through the stock ledger (`reason='sale'`), stamp the open `shift_id`, write the `sales` header and `sale_items` snapshots.
9. After commit, raise low-stock notifications and push SSE `low-stock` events for any product that crossed its threshold.

Sale record

Field	Meaning
`reference`	BIR-style serial, unique per tenant
`subtotal_cents / vat_cents / total_cents`	Net-of-VAT, output VAT, gross (VAT-inclusive)
`discount_cents / discount_label`	Total discount (incl. points) and its description
`payment_method / payment_ref`	Settlement channel and e-wallet reference
`tendered / change`	Cash handling
`shift_id / customer_id`	Links to drawer shift and CRM customer
`points_earned / points_redeemed`	Loyalty movements
`status / kind / reverses_sale_id`	Lifecycle and contra-entry linkage (section 10)

10. Voids, Returns & POS Audit Log

Contra-entry model

A void or return never edits or deletes the original sale. It writes a **new** `sales` row with `kind='void'` or `kind='return'`, negative quantities in `sale_items` (linked by `reverses_sale_item_id`) and a link `reverses_sale_id`. Every revenue total therefore nets automatically. The original's `status` moves to `voided`, `partially_returned` or `returned`. Reversal documents carry their own serial from `reversal_counters`.

- `GET /api/v1/pos/sales/:id` shows per-line **returnable quantity** (sold − already returned).
- `POST /api/v1/pos/sales/:id/void` reverses the whole sale; `POST …/returns` reverses chosen lines and quantities, with a reason.
- Stock is restored through the ledger (`sale_void` / `return`); loyalty points earned are clawed back and redeemed points restored proportionally.

POS audit log

Sensitive till actions are recorded in `pos_audit_logs` via `POST /api/v1/pos/audit`: `void_item` (line removed before payment), `cancel_transaction` (cart abandoned) and `open_drawer` (no-sale drawer open). Each row snapshots the cashier name, item, quantity, peso value and detail. Owners review it in **Audit log** (`/dashboard/audit`).

11. Cashier Shifts (X-Read / Z-Read)

The drawer ledger is keyed to the operator: **one open shift per cashier per tenant** (partial unique index). Sales are stamped with their `shift_id`, so tallies derive from real transactions.

Endpoint	Behaviour
`POST /api/v1/pos/shifts/open`	Opens a shift with the opening cash float (`openingCents`; a per-cashier default float can be configured). `409` if one is already open.
`GET /api/v1/pos/shifts/active`	**X-Read**: live tallies — cash, e-wallet and card revenue, transaction count, and expected drawer cash = opening float + cash sales.
`POST /api/v1/pos/shifts/close`	**Z-Read**: locks the shift, snapshots per-channel rollups, computes **variance to the centavo** (counted − expected; + overage / − shortage), freezes an immutable audit snapshot. Body `{countedCents, note?}`.
`POST /api/v1/staff/cashiers/:id/close-shift`	Owner force-closes a cashier's forgotten shift.

An un-reconciled variance raises a **security flag**: a notification to the owner (if `notify_variance` is on) and an `alert` event on the Super Admin health stream. HR's **Shift Discrepancy Matrix** lists reconciliations (`/api/v1/hr/shift-reconciliations`).

12. Inventory & Stock Ledger

Catalog

- **Products**: name, SKU (unique per tenant), price (pesos in, stored as centavos), stock, low-stock threshold, category, active flag, optional image (≤ 2 MB, `image/*`). Derived `lowStock` flag.
- **Categories**: nested via self-referential `parent_id`; unique name per tenant. Deleting a category makes its products uncategorised.
- UI: **Inventory Matrix**, product form with image picker, category manager.

Immutable stock-movement ledger

Every on-hand change goes through `moveStock()` and writes a signed, explainable row in `stock_movements` with `balance_after` (never negative), the **reason** and a reference to the source document.

Reason	Source	Sign
`sale`	Checkout	−
`sale_void`, `return`	Void / return	+
`po_receive`	Purchase order received	+
`produce_consume` / `produce_output`	Production run	− / +
`adjust`, `count`	Manual edit / stock count	±

- `GET /api/v1/inventory/products/:id/movements` — one product's history.
- `GET /api/v1/inventory/movements` — tenant-wide movement feed.
- `GET /api/v1/inventory/ledger-drift` — integrity check: products whose cached `stock` disagrees with the ledger's latest balance.
- `GET /api/v1/inventory/alerts` — low-stock engine: active products at or below threshold, most urgent first.

13. Procurement & Supply Chain Business

- **Suppliers**: name, contact person, e-mail, phone, address, note, active flag — CRUD at `/api/v1/procurement/suppliers`.
- **Purchase orders**: per-tenant serial reference (`po_counters`), supplier, line items (product, qty, unit cost), total, status `draft → ordered → received` or `cancelled`.
- **Receiving** (`POST /purchase-orders/:id/receive`) restocks every line in one transaction via the ledger (`po_receive`) and records the unit cost used for inventory valuation.
- **Reorder suggestions** (`GET /reorder-suggestions`): low-stock or depleted active products, a suggested quantity (pure `suggestReorderQty`) and the last known purchase cost (single-scan `DISTINCT ON` query, indexed).
- **Reorder modal**: tick items, adjust quantities and create one draft PO in a click.
- Summary KPIs at `/summary`; PO detail drawer in the UI.

14. Manufacturing (Bill of Materials) Enterprise

- **Recipes**: an output product, output quantity per batch and components (raw-material products and quantity consumed per batch).
- **Produce** (`POST /recipes/:id/produce` with `batches`): in one transaction, checks component availability, consumes components (`produce_consume`), adds output (`produce_output`), and writes a serialized `production_runs` record with its item breakdown.
- Production history at `/runs`; summary KPIs at `/summary`.
- Typical use: a café turning beans, milk and syrup into finished drinks or packaged goods.

15. Finance & Accounting Business

Expenses

CRUD at `/api/v1/finance/expenses`: date incurred, category, payee, amount (centavos), payment method (`Cash, GCash, Maya, Bank Transfer, Card, Other`), note.

Profit & Loss

`GET /finance/summary` — revenue (net of voids/returns), output VAT, discounts, expenses by category, net profit and trend charts.

Balance Sheet — `GET /finance/balance-sheet`

Line	Derivation
Cash & equivalents	Σ sales collected − Σ expenses − Σ goods received (a derived proxy, not a bank balance)
Inventory	stock × latest purchase unit cost
Output VAT payable	Cumulative VAT on sales
Accounts payable	Open purchase orders
Owner's equity	Assets − Liabilities (residual) → **Assets ≡ Liabilities + Equity** by construction

Cash Flow — `GET /finance/cash-flow?month=YYYY-MM`

Direct method for one Manila month: opening cash (cumulative strictly before the month), inflows (sales collections), outflows (expenses + inventory received), net change, closing cash. Invariants: `Opening + Net = Closing` and `Inflows − Outflows = Net`.

> Reports only sum pre-rounded integer centavos and never multiply by a tax rate, so they cannot introduce rounding drift. Builders live in the pure, unit-tested `finance.reports.ts`.

16. Human Resources & Payroll Enterprise

Employees

Name, position, employment type (`Full-time, Part-time, Contract`), pay type (`Monthly, Daily, Hourly`), pay rate, hire / separation dates, PTO balance, bank details, address, emergency contact, optional link to a cashier login. Profile page at `/dashboard/hr/employees/[id]`.

Attendance & timecards

- **Attendance** (`PUT /hr/attendance`): one row per employee per day — `Present, Absent, Leave, Half-day` with hours. Calendar view.
- **Timecards** (`/api/v1/merchant/shifts/clock-in|clock-out|status|me`): real punches from the POS clock widget; one active timecard per user.
- **Employee Self-Service** (`/me`, `/api/v1/ess/me`): staff view their own clock records.

Payroll runs

`POST /hr/payroll` computes gross for a period in one transaction with a per-tenant serial: Monthly → flat rate; Daily → rate × (Present days + ½ Half-days); Hourly → rate × hours logged. Statutory deductions (SSS, PhilHealth, Pag-IBIG, withholding tax) are out of scope for v1.

Automated payroll exporter

`GET /hr/payroll/export[/preview]` turns timecards into accounting-ready CSV using a pure processor in Manila time:

Component	Rule
Regular hours	Up to 8 worked hours per Manila day
Overtime	Hours beyond 8 per day at **125%**
Night differential	Hours between 22:00 and 06:00 at **+10%** (toggleable)
Monthly-to-hourly	Monthly rate ÷ 176 hours (22 days × 8 h)

Analytics

HR overview, **Performance Matrix** (sales per cashier), **Labor Analytics** (labor cost vs. revenue) and **Shift Discrepancy Matrix**.

17. Customer Relationship Management Business

- Customers: name, phone, e-mail, address, tags, note, active flag and loyalty points balance; CRUD at `/api/v1/crm/customers`.
- `/customers/search` powers the POS attach-customer lookup (cashiers have access).
- Customer detail drawer: purchase history, lifetime spend, points earned and redeemed.
- Loyalty rules: **earn 1 point per ₱100** of net paid; **redeem 1 point = ₱1**; returns reverse points.
- Summary KPIs (`/summary`): customer count, active members, outstanding points.

18. Merchant Dashboard, Analytics & Forecasting

- **Live Pulse** (`GET /api/v1/merchant/analytics/dashboard`): today's gross, transaction count, AOV, units sold, net / VAT / discount totals, hourly revenue velocity, payment-channel mix and best sellers — all real SQL aggregates, "today" in Manila time.
- **Owner Summary** (`/analytics/summary`, page `/dashboard/summary`): period rollups.
- **Predictive inventory** `Enterprise`: a pure forecast engine computes per-product sales velocity over a window, projects days of cover and flags at-risk items with suggested order quantities.
- Low-stock panel live-merged from the SSE stream; trial recovery, compliance tray and tier badge components.

Merchant navigation

Group	Items
Operations	Dashboard, Summary, Point of Sale, Inventory, Checkout QR, Compliance
ERP (tier-gated)	Finance, Procurement, Manufacturing, HR, CRM
Administration	Cashiers, Audit log, My record, Account

19. BIR Compliance

- **Serialized invoices**: immutable, gap-free, chronological `SI-` numbers per tenant, assigned under a row lock (no `count(*)+1` collisions between simultaneous tills).
- **12% VAT** price-inclusive, computed on the net after discounts and frozen on the sale.
- **Receipt fields**: registered name, business style, address, TIN, VAT label (e.g. "VAT REG TIN"), invoice prefix, header and footer text, logo toggle, and machine-accreditation footer — **PTU No.**, **MIN** and machine **serial**.
- **Monthly sales ledger**: `GET /api/v1/merchant/compliance/export?month=YYYY-MM` — invoices with gross, net taxable, output VAT, discounts and e-wallet references; JSON for the on-screen audit tray or `&format=csv` for tax preparation.
- **Platform compliance register** for the Super Admin: cross-tenant summary and consolidated CSV.
- Shift X-Read / Z-Read with centavo variance.

20. Account, Store Settings & Theme Studio

Page `/dashboard/account`, API `/api/v1/account/*` (owner / manager).

Card	Function
Store profile	Name, address, phone, e-mail, business hours, TIN, business style, logo upload
Receipt settings	Header/footer, VAT label, invoice prefix, show logo, PTU / MIN / serial
Theme Studio **Business**	Pick a preset vibe (`minimalist_apparel`, `espresso_lounge`, `fast_track`) and customise primary colour, surface, paper, ink, font (sans / mono / serif), product aspect ratio (1:1 or 4:5) and hero media. Stored as validated JSONB design tokens — never raw CSS — and applied to the back office, POS and customer display.
Profile	Name, phone, avatar
Preferences	Theme (light / dark / system); low-stock, variance and daily-summary notifications
Security	Password, active sessions (revoke one / all others), login activity
Data export	CSV of `sales`, `inventory` or `customers`
Danger zone	Owner-only store deactivation

Checkout QR (payments)

`/dashboard/payments`: upload or remove the store's static QR for **GCash**, **Maya** and **QR Ph** (`/api/v1/merchant/payment-qrs/:method`), shown at checkout.

Cashiers (owner only)

`/dashboard/staff`: create, edit, disable cashiers, set PINs, set default float, resolve forgot-PIN requests, force-close shifts.

21. Notifications & Real-Time Streams

- **In-app notifications** (`notifications` table): severity `info / success / warning / danger`, unread de-duplication index. Bell with unread count; mark one or all read.
- **Producers**: low stock after a sale, shift variance, PIN reset requests, trial events, and more — respecting each user's preferences.
- **Merchant SSE** `/api/v1/merchant/events/stream`: `low-stock` events (3-second tick) turned into warning toasts.
- **Admin SSE** `/api/v1/admin/health/stream`: `metrics`, `activity`, `alert` events.
- **E-mail** (Resend): onboarding, trial-expired recovery and enterprise-lead alerts.

22. Super Admin Platform Console

Page	Function
`/admin`	Metrics bento overview, platform shortcuts
`/admin/leads`	Leads Matrix: pipeline `PENDING_DEMO / APPROVED / REJECTED`; approve (atomic provisioning with store name, slug, plan, owner) or reject
`/admin/tenants`	Tenants Directory: all stores, plan, status, owner; change plan/status; impersonate owner
`/admin/billing`	MRR engine: MRR, active subscribers, ARPA, annual run rate, per-tier breakdown, six-month fee trend and cumulative MRR growth; Subscribers drawer
`/admin/health`	Health Matrix: NeonDB pool occupancy, query round-trip latency, cross-tenant activity ticker, shift-variance alerts
`/admin/compliance`	Cross-tenant BIR compliance register and consolidated CSV

23. Marketing Website & Lead Capture

Public pages: home (hero, problem, core tools, how it works, comparison vs. Square and Lightspeed, testimonials, pricing, FAQ, why-PH, sticky CTA), `/products/[slug]`, `/industry/[slug]`, `/hardware/[slug]`, `/enterprise` (concierge), `/company/[slug]`, `/about`, `/careers`, `/blog/[slug]`, `/resources`, `/security`, `/contact`, `/terms`, `/privacy`, and stateless previews `/preview/merchant` and `/preview/pos`.

- `POST /api/leads` — demo request `{name, businessName, email, phone, businessType, message?}`; upsert on e-mail; `400 {errors}` on invalid input.
- `POST /api/leads/enterprise` — enterprise concierge request; e-mails the sales alert.
- `GET /api/v1/public/tenants` — public directory of stores (trust strip).

24. Hardware Integration

Device	Integration
Barcode / QR scanner	Any keyboard-wedge (USB/Bluetooth HID) scanner. `useBarcodeScanner` captures rapid Enter-terminated bursts at window level and distinguishes them from human typing by inter-key timing.
Thermal receipt printer	Pure ESC/POS serialiser (`lib/escpos.ts`) + transports (`lib/thermalPrinter.ts`): **WebUSB** for desk printers, **Web Bluetooth** for mobile printers. Money printed as "PHP 1,234.56" (ASCII-safe). Falls back to the HTML receipt on unsupported browsers.
Customer display	Second monitor or tablet on `/pos/customer-display`, synced over `BroadcastChannel`.
Cash drawer	Drawer-open events logged in the POS audit log.

> WebUSB and Web Bluetooth require a Chromium-based browser (Chrome, Edge) over HTTPS or localhost.

25. Database Schema Reference

PostgreSQL (NeonDB). All primary keys are UUIDs (`gen_random_uuid()`); every tenant-owned table carries `tenant_id … ON DELETE CASCADE`. The schema is applied by the idempotent `npm run migrate`.

Table	Purpose & key columns / constraints
`tenants`	name, slug, plan (starter/business/enterprise), tier, status (trial/active/trial_expired/suspended), trial dates, store profile, TIN, business style, logo, theme_color, theme_config JSONB, receipt fields (header, footer, VAT label, prefix, PTU, MIN, serial)
`users`	tenant_id, email (unique, case-insensitive), name, role, status, PIN hash, password hash, default float, phone, avatar, theme, notification prefs. Check: SUPER_ADMIN ⇔ no tenant
`sessions`, `login_events`	Revocable device sessions; login audit trail
`leads`	Demo requests; status PENDING_DEMO/APPROVED/REJECTED; unique email
`categories`, `products`	Nested categories; products with SKU (unique per tenant), price_cents, stock ≥ 0, low_stock_threshold, image_url, active
`stock_movements`	Immutable ledger: qty_delta, balance_after ≥ 0, reason, ref_table/ref_id, note, created_by
`sales`, `sale_items`	Sale headers and line snapshots; status, kind (sale/void/return), reverses links, shift, customer, points; qty ≠ 0
`invoice_counters`, `reversal_counters`	Per-tenant locked serial counters
`cashier_shifts`	Opening/expected/counted cash, variance, channel rollups, audit snapshot; one open per user
`timecards`	Clock-in/out punches; ACTIVE ⇔ no clock_out; one active per user
`cashier_pin_requests`	Forgot-PIN requests; one pending per cashier
`pos_audit_logs`	void_item / cancel_transaction / open_drawer events
`tenant_payment_qrs`	Static QR per method (GCash, Maya, QRPH)
`expenses`	Category, amount, date, payment method
`suppliers`, `purchase_orders`, `purchase_order_items`, `po_counters`	Procurement; PO status draft/ordered/received/cancelled; unit costs
`recipes`, `recipe_components`, `production_runs`, `production_run_items`, `production_counters`	Manufacturing / BOM
`employees`, `attendance`, `payroll_runs`, `payroll_items`, `payroll_counters`	HR and payroll; attendance unique per employee per day
`customers`	CRM with loyalty_points ≥ 0

`notifications`	Per-user notifications with severity and unread de-dupe

Notable indexes

`sales (tenant_id, created_at DESC)`, `sales (tenant_id, lower(reference))` unique, `sales_status_idx`, `stock_movements` by tenant / product / ref, `purchase_order_items_product_idx` (reorder and balance-sheet cost lookup), partial unique indexes for one open shift, one active timecard and one pending PIN request.

26. Complete API Reference

Roles: SA = SUPER_ADMIN, OW = MERCHANT_OWNER, MG = MANAGER, CA = CASHIER. Tier tags show the minimum plan.

PUBLIC & AUTHENTICATION

Endpoint	Access	Purpose
`GET /health`	public	Liveness
`GET /uploads/*`	public	Uploaded media
`POST /api/leads` · `POST /api/leads/enterprise` · `GET /api/leads`	public	Lead capture / list
`GET /api/v1/public/tenants`	public	Public store directory
`GET /auth/google` · `GET /auth/google/callback`	public	Google OAuth
`GET /auth/me` · `POST /auth/logout`	any session	Current session / sign out
`POST /auth/password-login` · `POST /auth/signup`	public	Password login; self-serve sign-up
`GET /auth/password/status` · `POST /auth/password`	OW, MG	Password management
`POST /auth/impersonate` · `POST /auth/impersonate/stop`	SA	Support impersonation
`POST /auth/store` · `/auth/pin` · `/auth/cashiers` · `/auth/pin/forgot`	public	Store resolve, PIN login, cashier list, forgot-PIN

POS — `/API/V1/POS` (CA, OW, MG)

Endpoint	Purpose
`GET /catalog` · `GET /cashiers`	Register feed; cashier names
`POST /orders`	ACID checkout
`GET /sales` · `GET /sales/:id`	Sales history / detail with returnable qty
`POST /sales/:id/void` · `POST /sales/:id/returns`	Contra-entry void / return
`POST /audit` · `GET /audit`	POS audit log
`GET /shifts/active` · `POST /shifts/open` · `POST /shifts/close`	X-Read, open, Z-Read

INVENTORY — `/API/INVENTORY` AND `/API/V1/INVENTORY` (OW, MG)

Endpoint	Purpose
`GET|POST /products` · `PATCH|DELETE /products/:id`	Products (multipart image; `removeImage=true`)
`GET|POST /categories` · `PATCH|DELETE /categories/:id`	Categories
`GET /v1/inventory/alerts`	Low-stock alerts
`GET /v1/inventory/products/:id/movements` · `/movements` · `/ledger-drift`	Stock ledger

MERCHANT — `/API/V1/MERCHANT` (OW, MG UNLESS NOTED)

Endpoint	Purpose
`GET /analytics/dashboard` · `/analytics/summary`	Live Pulse; owner summary
`GET /analytics/forecast` **Ent**	Predictive inventory
`GET /compliance/export?month=&format=csv`	BIR sales ledger
`GET /inventory/alerts` · `GET /events/stream`	Alerts; SSE
`GET /payment-qrs` · `POST|DELETE /payment-qrs/:method`	Checkout QRs
`GET /shifts/status` · `POST /shifts/clock-in|clock-out` · `GET /shifts/me` (CA, MG, OW)	Timecards

STAFF — `/API/V1/STAFF` (OW ONLY)

Endpoint	Purpose
`GET|POST /cashiers` · `PATCH|DELETE /cashiers/:id`	Cashier roster
`POST /cashiers/:id/pin` · `POST /cashiers/:id/close-shift`	Set PIN; force-close shift
`POST /pin-requests/:id/reject`	Dismiss forgot-PIN request

FINANCE — `/API/V1/FINANCE` (OW, MG) BUSINESS

`GET /summary` · `/balance-sheet` · `/cash-flow?month=`	P&L and statements
`GET|POST /expenses` · `PATCH|DELETE /expenses/:id`	Expenses

PROCUREMENT — `/API/V1/PROCUREMENT` (OW, MG) BUSINESS

`GET /summary` · `/reorder-suggestions`	KPIs; reorder
`GET|POST /suppliers` · `PATCH|DELETE /suppliers/:id`	Suppliers
`GET|POST /purchase-orders` · `GET|DELETE /purchase-orders/:id`	Purchase orders
`POST /purchase-orders/:id/receive` · `PATCH /purchase-orders/:id/status`	Receive; status change

MANUFACTURING — `/API/V1/MANUFACTURING` (OW, MG) ENTERPRISE

`GET /summary` · `GET /runs`	KPIs; production history
`GET|POST /recipes` · `GET|PATCH|DELETE /recipes/:id`	Recipes / BOM
`POST /recipes/:id/produce`	Production run

HR — `/API/V1/HR` (OW, MG) ENTERPRISE

`GET /summary` · `/overview` · `/performance` · `/labor-analytics` · `/shift-reconciliations`	Analytics

`GET|POST /employees` · `GET|PATCH|DELETE /employees/:id`	Employees
`GET|PUT /attendance`	Attendance
`GET|POST /payroll` · `GET /payroll/:id` · `GET /payroll/export[/preview]`	Payroll runs and CSV export
`GET /api/v1/ess/me` (any staff)	Employee self-service

CRM — `/API/V1/CRM` (OW, MG, CA) BUSINESS

`GET /summary` · `GET /customers` · `GET /customers/search`	KPIs, list, POS lookup
`GET|PATCH|DELETE /customers/:id` · `POST /customers`	Customer CRUD

ACCOUNT — `/API/V1/ACCOUNT` (OW, MG)

`GET|PATCH /store` · `POST|DELETE /store/logo`	Store profile
`PATCH /theme` · `GET|PUT /theme-config` Business	Theme Studio
`GET|PATCH /receipt`	Receipt settings
`GET|PATCH /profile` · `POST|DELETE /profile/avatar` · `GET|PATCH /preferences`	Profile & preferences
`GET /sessions` · `DELETE /sessions/:id` · `POST /sessions/revoke-others` · `GET /login-events`	Sessions & login audit
`GET /export/:dataset` · `POST /deactivate` (OW)	CSV export; deactivate store

NOTIFICATIONS — `/API/V1/NOTIFICATIONS` (SIGNED-IN)

`GET /` · `GET /unread-count` · `POST /:id/read` · `POST /read-all`	Notification bell

SUPER ADMIN — `/API/V1/ADMIN` (SA)

`GET /leads` · `POST /leads/:id/approve` · `POST /leads/:id/reject`	Lead pipeline & provisioning
`GET /tenants` · `PATCH /tenants/:id`	Tenant directory
`GET /analytics/billing` · `GET /analytics/subscribers`	MRR engine
`GET /health/metrics` · `GET /health/stream`	System health
`GET /compliance/summary` · `GET /compliance/export`	Compliance register

STANDARD RESPONSES

`200/201 {ok:true,…}` · `400 {ok:false,errors}` validation · `401` no session · `402` trial expired · `403` wrong role, tier (feature locked) or `plan_limit_exceeded` · `404` not found / other tenant · `409` conflict (duplicate, open shift, slug collision).

27. Frontend Route Map

Route	Audience	Purpose
`/` and marketing pages	Public	Landing, products, industries, hardware, enterprise, company, blog, legal
`/login`	All staff	Google, owner password, Store ID → PIN
`/auth/signup`	Prospects	Self-serve 14-day trial sign-up
`/pos`	CA, MG, OW	POS terminal
`/pos/customer-display`	Customer	Second screen
`/dashboard`	OW, MG	Merchant home (Live Pulse)
`/dashboard/summary`, `/inventory`, `/payments`, `/compliance`	OW, MG	Core operations
`/dashboard/finance`, `/procurement`, `/manufacturing`, `/hr`, `/hr/employees/[id]`, `/crm`	OW, MG (tier-gated)	ERP modules
`/dashboard/staff`, `/audit`, `/account`	OW (staff), OW/MG	Administration
`/me`	Any staff	Employee self-service clock records
`/admin`, `/admin/leads`, `/tenants`, `/billing`, `/health`, `/compliance`	SA	Platform console
`/preview/merchant`, `/preview/pos`	Public	Stateless product previews

28. Security Model

- **Tenant isolation**: tenant id comes only from the signed session and is threaded into every query; cross-tenant ids resolve as not found.
- **Defence in depth**: role guard → tier guard → trial checkpoint → zod validation → parameterised SQL (no string-built queries) → DB check constraints.
- **Credentials**: scrypt-hashed PINs and passwords with per-secret salts; constant-time comparison; PIN hashes never leave the DB.
- **Sessions**: HTTP-only cookie, CSRF `state` on OAuth, revocable sessions, login audit, 7-day expiry.
- **Transport**: helmet headers, explicit CORS allow-list with credentials, 100 kB JSON cap, 2 MB image cap with MIME filter.
- **Integrity**: immutable stock ledger, contra-entry reversals, locked serial counters, frozen Z-Read snapshots, POS audit log.
- **Infrastructure**: managed NeonDB (encryption at rest, backups) and HTTPS hosting.

29. Installation & Configuration

Prerequisites

Node.js 20+ (built with v22) and a NeonDB project (`console.neon.tech`).

Backend

```
cd backend
npm install
cp .env.example .env        # paste the Neon DATABASE_URL
npm run migrate             # idempotent schema
npm run dev                 # http://localhost:4000
# optional
npm run seed:owner-passwords
npm run validate:reports    # staging report harness
npm test                    # vitest
```

Frontend

```
cd frontend
npm install
cp .env.example .env.local  # NEXT_PUBLIC_API_URL=http://localhost:4000
npm run dev                 # http://localhost:3000
npm run build && npm start  # production
```

Environment variables — backend

Variable	Purpose
`DATABASE_URL`	Neon connection string (required)
`PORT`	API port (default 4000)
`CORS_ORIGIN`	Comma-separated allowed origins
`FRONTEND_URL`	Post-login redirect base
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	OAuth web client (blank = Google sign-in disabled)
`GOOGLE_REDIRECT_URI`	e.g. `http://localhost:4000/auth/google/callback`
`JWT_SECRET`	Long random string; signs sessions
`SESSION_COOKIE_NAME` / `COOKIE_DOMAIN`	Cookie name (default `vendopos_session`) and optional shared domain
`PUBLIC_BASE_URL`	Origin used to build upload URLs
`RESEND_API_KEY` / `EMAIL_FROM`	Transactional e-mail (optional)

Environment variables — frontend

`NEXT_PUBLIC_API_URL`	Base URL of the Express API

Bootstrapping the first Super Admin

```
INSERT INTO users (email, name, role) VALUES ('you@gmail.com', 'You', 'SUPER_ADMIN');
```

Then sign in with Google using that e-mail. Merchants are created through Lead approval or self-serve sign-up.

30. Testing & Quality Assurance

- **Backend**: vitest + supertest, ~288 test cases across every module (auth, tier middleware, trial, checkout, refunds, shifts, audit, stock ledger, finance reports, reorder, forecast, payroll processor, timecards, HR, CRM, manufacturing, procurement, admin, billing, health, payment QRs, theme config, e-mails). Validation seams are tested without a database.
- **Pure cores**: `finance.reports`, `procurement.reorder`, `forecast`, `payroll.processor`, `theme.config`, `tiers` are IO-free and fully unit-tested.
- **Static checks**: `tsc --noEmit` (backend and frontend), ESLint, `next build` prerenders all routes.
- **Visual**: Playwright screenshot scripts (`frontend/__shots`) for light/dark modals, POS and customer display.

31. Staging Validation Runbook

`backend/src/validateReports.ts` (`npm run validate:reports`) exercises the real HTTP runtime against a deployed environment, sweeps every active tenant via Super Admin impersonation, asserts accounting invariants and times every call. It is **read-only** and always exits 0 (report only).

Run

```
BASE_URL=https://staging.vendopos.example \
ADMIN_SESSION_COOKIE=<vendopos_session value> \
SLOW_MS=200 CASHFLOW_MONTHS=12 \
npm run validate:reports
```

Copy the cookie value from DevTools → Application/Storage → Cookies (it is HTTP-only, so `document.cookie` will not show it).

Variable	Default	Purpose
`BASE_URL`	http://localhost:4000	API origin
`ADMIN_SESSION_COOKIE`	required	Super Admin session value
`SLOW_MS`	500	Flag slow calls (⚠)
`CASHFLOW_MONTHS`	3	Months of cash flow per tenant

Result codes

✓ **ok** — invariants hold; ✗ **FAIL** — invariant broke or non-200 (investigate before production); **– skip** — no active owner to impersonate; ⚠ — slowest call above threshold (confirm `npm run migrate` created `purchase_order_items_product_idx`).

Invariants checked

- Balance Sheet: `Assets ≡ Liabilities + Equity` — a failure is a code/data defect, not a bookkeeping quirk.
- Cash Flow: `Opening + Net ≡ Closing` and `Inflows − Outflows ≡ Net` for every month checked, including months before the tenant existed (all zero).

32. Capability Assessment & Roadmap

Benchmarked against the *Comprehensive Guide to Cloud ERP & Integrated POS Software* (gap analysis 2026-06-13):

> **Verdict:** over-qualified for single-location Philippine SMBs; qualified-with-gaps for multi-branch chains and batch/expiry-regulated verticals (pharmacy, grocery, food).

Delivered

- All five ERP pillars wired to the POS: Finance, Procurement, Manufacturing, HR, CRM.
- Balance Sheet and Cash Flow (A1), barcode scan-to-cart (A3), low-stock → draft PO reorder (A4).
- Immutable stock ledger, contra-entry voids/returns, gap-free BIR invoicing, Z-Read variance flags.
- Billing/MRR, trials, health telemetry, notifications, revocable sessions, login audit, Theme Studio, thermal printing, customer display.

Open items

#	Item	Status / notes
A2	Batch / lot tracking + expiry (FIFO)	Deferred — proposed `product_batches` table, receiving stamps batch + expiry, near-expiry alerts
B1	Sales commissions in payroll	Needs decision — opt-in separate payroll line
B2	Multi-location / inter-branch transfers	Invasive (`location_id` across stock and sales); separate effort. Feature key `multi_location` is declared
—	Offline register mode	Feature key `offline_mode` is declared and marketed; no offline queue/sync is implemented yet
—	Statutory payroll deductions	SSS, PhilHealth, Pag-IBIG, withholding tax not yet computed
—	Subscription payment collection	MRR is reported; card / e-wallet charging is not yet wired
—	Cloud object storage	Uploads use local disk; swap the storage provider for S3 / Supabase / Cloudinary in production
—	Price alignment	Marketing (₱599 / ₱1,999) vs. billing engine (₱499 / ₱1,499)
C	Biometric attendance, controlled-substance register, weight pricing, promotions engine, wholesale tiers	Vertical-specific; not core

— End of document —